



summarize their data pipeline

Their pipeline is: Binance tick data → bar construction → feature engineering → label generation → train/validation/test splits → model training → trading backtest.^[1]

The key twist is that they replace fixed time bars with information-driven bars, especially CUSUM, and replace next-bar labels with Triple Barrier labels.^[1]

Pipeline

- They start with Binance BTC and ETH tick-level data from Jan 2018 to Jun 2023.^[1]
- They aggregate raw ticks into OHLCV bars using either standard time bars (1h, 12h, 24h) or information-driven bars: volume bars, dollar bars, range bars, and CUSUM bars.^[1]
- After sampling, they compute features on the bars: OHLCV plus technical indicators like EMAs, MACD, RSI, stochastic oscillator, Williams %R, Bollinger Bands, returns, CMF, MFI, and cyclical hour/weekday features.^[1]
- They scale all features by subtracting the mean and dividing by the standard deviation before training.^[1]

Labels and training

- They create targets with either next-bar direction or the Triple Barrier method, which uses take-profit, stop-loss, and a time limit.^[1]
- They use a quarterly expanding-window split, training on older data and testing on five out-of-sample quarters from Q2 2022 to Q2 2023.^[1]
- They train several models, tune hyperparameters on validation data, use early stopping, and ensemble the top three variants of each model for test predictions.^[1]

Backtest

- They convert predictions into long/short trades, include 0.1% entry and 0.1% exit transaction costs, allow only one open position at a time, and ignore weak signals near 50% confidence.^[1]
- They then evaluate profit, Sharpe ratio, drawdown, accuracy, and hit rate, and find the best overall pipeline is CUSUM sampling plus Triple Barrier labeling, usually with ResNet-LSTM.^[1]

help me understand cusum filtering

CUSUM filtering is a way to detect meaningful moves instead of sampling on a fixed clock. It keeps a running sum of deviations from a reference level and triggers only when that accumulated move gets large enough.^{[2] [3] [4]}

Intuition

Think of it as a noise filter for price moves. Small back-and-forth changes tend to cancel out or reset, but a persistent move in one direction builds up the cumulative sum until it crosses a threshold.^{[5] [3] [2]}

How it works

A standard positive CUSUM updates a statistic like $S_t = \max(0, S_{t-1} + x_t - \omega_t)$, and it signals an event when S_t passes a chosen threshold; a mirrored negative version detects downward shifts.^[2]

In practice, you choose a baseline or target, a drift term k that absorbs small moves, and a threshold h that decides when the move is large enough to matter.^{[3] [5]}

In this paper

Here, CUSUM is used as an information-driven sampling method, so the model gets a new bar when market conditions have changed enough, not every hour or day.^[4]

That matters in crypto because trading is continuous and volatile, so event-based sampling can react faster than fixed time bars to real market activity.^[4]

Why traders like it

- Fewer observations from pure noise, more from real moves.^{[3] [4]}
- Decisions can happen at any time, not only at fixed intervals.^[4]
- It is especially good at detecting small but persistent shifts.^{[6] [2]}

A one-line version: CUSUM asks, “Has price moved far enough, consistently enough, that I should care yet?”^{[2] [3]}

✱

our supervisor told us to research if we could sample more the closer we get to the trade. How does that work?

Yes — that usually means adaptive resolution: use coarse data far from the decision, then finer data as you approach entry, so the model sees more detail when timing matters most.^{[13] [14]}

It is the same core idea as information-driven bars: sample more often when the market is carrying more information, and less often when it is quiet.^{[15] [14] [13]}

What it means

In the paper, their CUSUM and other information-driven bars already do a version of this by creating more bars during active periods and fewer during quiet periods, instead of sampling every fixed hour.^[14]

So “closer to the trade” can mean either closer in **time** to entry, or closer in **market state** to a trigger, such as rising volatility or a cumulative move crossing a threshold.^{[13] [14]}

How to implement it

A practical setup is multi-resolution input:

- Far away: low-frequency bars for regime context.
- Mid-range: medium-frequency bars for setup formation.
- Right before entry: high-frequency or CUSUM/event bars for timing.

Example: last 3 days in 1h bars, last 6 hours in 5m bars, last 30 minutes in tick/CUSUM bars. That gives you broad context plus precise entry timing without processing every tick all the time.

Two main versions

- Event-driven sampling: keep one stream, but generate bars faster when price/volume activity rises, which is exactly why CUSUM is useful.^{[15] [14] [13]}
- Time-to-event refinement: once a potential trade setup appears, switch to finer sampling only for that local window.

The second version is probably what your supervisor means.

Important caveat

Do not decide “closer to the trade” using future information.

The refinement rule must be triggered only by data available at that moment, otherwise you leak label information into the features.

A good thesis framing is: “Use coarse sampling for state detection and fine sampling for execution.”

✱

what is OHLC in the paper?

In the paper, OHLC means Open, High, Low, Close prices for each bar.^[29]

So each sampled bar stores the price at the start of the interval, the highest price reached, the lowest price reached, and the price at the end.^[29]

In this paper

They say it has become standard to sample OHLC data at regular intervals like hourly or daily, and their contribution is to build those same kinds of bars with information-driven sampling instead of fixed clock time.^[29]

So the bar is still an OHLC bar, but the interval can be triggered by market activity rather than by waiting for the next hour.^[29]

Example

If a bar covers some event window, then:

- Open = first traded price in that bar.^[29]
- High = max price during that bar.^[29]
- Low = min price during that bar.^[29]
- Close = last traded price in that bar.^[29]

Usually volume is added too, so people often say OHLCV.^[29]

✱

Feature engineering is done after labelling right?

No — in this paper, feature engineering is tied to the sampled bars, not done “after labeling.”^[30] They explicitly say the technical indicators and time features were computed **after the sampling procedure**, and target labeling is described as a separate step.^[30]

Order

A good reading of their pipeline is:

1. Sample raw data into bars.^[30]
2. Compute features on those bars.^[30]
3. Generate labels, such as next-bar or Triple Barrier targets.^[30]

Why

Features should come from information available at or before prediction time, while labels usually depend on what happens after that point.^[30]

So conceptually, labeling is attached afterward to each feature row, even if both are created from the same sampled dataset.^[30]

In short: sampling → features → labels is the safer interpretation here.^[30]

✱

I know that its after sampling but the tripple barrier method is the trade signal, which i suppose is a product of the features?

Not quite. In the paper, the Triple Barrier method is a **labeling** method, not a feature-derived signal.^[31]

It uses the future price path after a given observation to decide whether the upper barrier, lower barrier, or time barrier is hit first, and that outcome becomes the target label.^[31]

Distinction

Features are inputs like OHLCV and technical indicators.^[31]

The Triple Barrier label is the supervised target the model tries to predict from those features.^[31]

So what is the signal?

The model takes features at time t and outputs a prediction of the Triple Barrier label.^[31]

That prediction is what becomes the trade signal in the backtest, not the Triple Barrier procedure itself.^[31]

Clean way to think about it

- Triple Barrier = how they define the “correct answer” for training.^[31]
- Features = what the model sees before making a decision.^[31]
- Predicted label = the actual long/short signal used for trading.^[31]

So yes, the trade signal is a product of the model applied to the features, but the Triple Barrier method is the rule used to create the training target from future outcomes.^[31]

✱

that makes sense. its because, tripple barrier is just based in the pure outswing of the instrument? not dependent on the feature

Yes — that is basically right.^[32]

The Triple Barrier label is determined by the future price path after an observation: whether price hits the profit barrier, stop-loss barrier, or time barrier first.^[32]

Key point

So it depends on the instrument’s subsequent move, not on the engineered features themselves.^[32]

Features are only the inputs used to predict that future outcome.^[32]

Clean framing

- Features: what you know now.^[32]
- Triple Barrier label: what the market does next relative to the three barriers.^[32]
- Trade signal: the model's prediction of that label from the features.^[32]

One nuance: the barrier distances can be set using things like volatility or investor-defined thresholds, so the labeling rule can be parameterized, but the realized label still comes from the actual future swing in price.^[32]

✱

make a rough sketch of the data pipeline in this project

Yes — a rough sketch is: raw Binance data → sampling into bars → feature engineering → target labeling → train/validation/test split → model training → trade simulation and evaluation.^[33]

Sketch

- Raw Binance BTC/ETH data, covering Jan 2018 to Jun 2023.^[33]
- Sample into bars: time bars, volume bars, dollar bars, range bars, or CUSUM bars.^[33]
- Build bar data such as OHLC and volume.^[33]
- Add engineered features: technical indicators, returns, and cyclical time features.^[33]
- Scale features before training.^[33]
- Create labels: either next-bar direction or Triple Barrier outcomes.^[33]
- Split with a quarterly expanding window into train, validation, and out-of-sample test periods.^[33]
- Train models such as ResNet-LSTM, Attention-LSTM, XGBoost, Transformer variants, and TSMixer.^[33]
- Convert predictions into long/short trades, include Binance transaction costs, and evaluate profit, Sharpe, drawdown, and accuracy.^[33]

One-line diagram

Binance ticks → bars → features → labels → split → train models → predict →
backtest → metrics^[33]

For your report

A slightly more precise version would be: Raw market data -> information-driven sampling -> OHLCV bars -> feature engineering -> Triple Barrier labeling -> model training -> trade signals -> backtest evaluation.^[33]

✱

```
# <q>Add engineered features: technical indicators, returns, and cyclical time features.
```

Yes. They compute extra features for each sampled bar series after the bars are constructed.^[34]

So once you have a bar sequence—time bars, CUSUM bars, volume bars, dollar bars, or range bars—they add indicators like EMAs, MACD, RSI, Bollinger Bands, returns, CMF/MFI, plus sine/cosine hour and weekday features on top of those bars.^[34]

What that means

Each bar becomes one row with base fields like OHLCV, and then extra engineered columns derived from the current and recent bars.^[34]

So yes, every constructed bar gets an associated feature vector, except the earliest bars where some rolling indicators may be unavailable because the lookback window is not yet filled.^[34]

Mental model

Think of it as:

```
sampled bars -> one row per bar -> attach rolling indicators to each row -> then attach label[34]
```

The important part is that the indicators are computed from the bar history, not from raw ticks directly, and the paper says these features were computed for all sampling techniques after sampling.^[34]

✱

“Scale features before training.” what does this mean?

It means you transform the numeric feature columns so they are on a similar scale before feeding them to the model.^[35] ^[36]

This stops large-magnitude features from dominating just because their numbers are bigger, and it often helps training behave better.^[36] ^[35]

In practice

For example, one feature might range from 0 to 1 while another ranges into the thousands; scaling makes them more comparable.^[37] ^[36]

Common choices are standardization, which makes a feature have mean 0 and standard deviation 1, or min-max scaling, which maps values into a range like 0 to 1.^[35] ^[36]

In your pipeline

In your project, this would mean scaling the engineered bar-level features before model fitting.^[38] ^[35]

And the safe way is to compute the scaling parameters on the training set, then apply the same transform to validation and test data.^[39]



```
# <q>Create labels: either next-bar direction or Triple Barrier outcomes.</q> put a bi
```

Yes — this is the step where each bar gets a **target** the model should learn to predict.^[46]

The paper compares two ways to define that target: next-bar direction and Triple Barrier labeling.^[46]

Next-bar label

For next-bar direction, you look at the current bar and then check whether the following bar moved up or down.^[46]

That means each row gets a simple label based on the immediate next price move, which is the conventional approach the paper contrasts against.^[46]

Triple Barrier label

For Triple Barrier, you start from a bar and place three future-exit rules: an upper profit barrier, a lower stop-loss barrier, and a vertical time limit.^[46]

You then watch the future path of price and label the sample by whichever barrier is hit first.^[46]

Why this matters

Next-bar labeling asks, “What happens on the very next step?”^[46]

Triple Barrier instead asks, “From this entry point, does price hit profit, stop, or time expiry first?”, which the paper says is closer to real trading with take-profit and stop-loss logic.^[46]

Mental picture

So for each constructed bar, the pipeline is roughly: features from current and past bars, then a future-based label attached afterward.^[46]

The model never sees the future path directly; it only sees the current feature vector and tries to predict the label created from that later outcome.^[46]



so this could be, surpass threshold of +2%, buy?

Almost — more precisely, it would be: from an entry bar, if price hits the **upper barrier** such as +2% before it hits the lower barrier or the time barrier, that sample gets a positive/long label.^[47] So “+2% reached first” is a **labeling outcome**, not automatically the live buy rule itself.^[47]

Better wording

A rough version is:

- Upper barrier = +2% from entry.^[47]
- Lower barrier = −2% from entry, or whatever stop you choose.^[47]
- Vertical barrier = max holding time.^[47]
- If +2% is hit first, label = buy/long class.^[47]

Important nuance

In the project, the model is trained on these labels and then later predicts them from features.^[47]

So the actual trading rule is more like: “model predicts upper-barrier-first with high enough confidence, therefore enter long,” not “whenever price surpasses +2%, buy.”^[47]

That last version would be backward-looking and too late, because the +2% move is what the label is trying to anticipate.^[47]

✱

so each bar is reconstructed into a tripple bar=

Not exactly.^[48]

Each sampled bar is **not** turned into three new bars; instead, each bar is treated as a candidate entry point, and three barriers are attached to it: profit-taking, stop-loss, and a time limit.^[48]

Correct picture

So for bar t , you keep the bar and its features, then look forward in time to see which of those three barriers is hit first.^[48]

That first hit determines the label for that bar.^[48]

Tiny sketch

```
bar_t -> attach upper barrier, lower barrier, vertical barrier -> inspect future path -> assign label[48]
```

So it is better to say “each bar gets Triple Barrier labeling” rather than “each bar is reconstructed into a triple bar.”^[48]

✱

that makes sense. you dont make the barriers at each bar, because not all bars pass the threshhold

Close, but slightly different.^[49]

You do define the three barriers for each candidate bar/entry***, then many bars simply end up not hitting the profit barrier first.^[49]

What happens

For each bar, you set:

- Upper barrier.
- Lower barrier.
- Vertical time barrier.^[49]

Then you look forward and ask which one is hit first.^[49]

A bar does not need to pass the profit threshold to be labeled; it can hit the stop-loss first or just reach the time barrier first.^[49]

Better phrasing

So the right phrasing is: every bar gets a Triple Barrier setup, but not every bar gets a positive/buy label.^[49]

The label depends on the first barrier breached, not on whether the upper threshold is reached at all.^[49]

✱

yes, because most of the bars will only satisfy the middle of the tripple barrier where you dont do trading?

Not necessarily.^[50]

Each bar gets one of three outcomes: upper barrier first, lower barrier first, or vertical time barrier first.^[50]

Key idea

The “middle” case is basically the **time-expiry** case, meaning price did not hit profit-taking or stop-loss before the maximum holding horizon.^[50]

That can be treated as a neutral/no-signal class in a classifier, but it is not guaranteed that most bars land there.^[50]

In this project

How many bars end at the vertical barrier depends on the chosen barrier widths and the market volatility.^[50]

The paper notes that if barriers are set too low, Triple Barrier can nearly collapse toward next-bar labeling, which implies the class mix changes with parameter choice.^[50]

Trading meaning

So your intuition is partly right: bars that hit the vertical barrier first often correspond to “nothing meaningful happened within the horizon.”^[50]

But the safer statement is: some bars become neutral/time-out cases, not “most” by definition.^[50]



can you find the original article about the tripple barrier method?

Yes — the original source people cite for the Triple Barrier Method is Marcos López de Prado’s book *Advances in Financial Machine Learning*, published by Wiley in 2018, rather than a standalone journal article.^[51] ^[52]

Source

The method is widely described as having been introduced in that book, and practitioner references point to Chapter 3 for the Triple Barrier labeling procedure.^[53] ^[54] ^[51]

Book page: <https://www.wiley.com/en-us/Advances+in+Financial+Machine+Learning-p-9781119482086>^[52]

Want the exact bibliographic citation in APA or BibTeX format?



find a good free source explaing the tripple barrier method

Yes — the best free explainer I found is William Santos’s post, “Algorithmic trading: triple barrier labelling.”^[81]

It explains the intuition clearly: three barriers, label by first hit, why it improves on fixed-horizon labeling, and it includes Python implementation details.^[81]

Best pick

- William Santos — “Algorithmic trading: triple barrier labelling”: clear conceptual explanation, compares it to fixed-time horizon labeling, and walks through an implementation and example labels.^[81]
<https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/>^[81]

Good backup

- QuantInsti — “Triple Barrier Method: Python | GPU | Nvidia”: good if you want a more practical, coding-oriented explanation with barrier definitions and implementation ideas.
<https://blog.quantinsti.com/triple-barrier-method-gpu-python/>

If you want just one, read William Santos first.^[81]

✱

<https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/>

create a bib section for this article

Use this @misc entry; it is the safest choice for a web article like the Triple Barrier explainer page.^[97]

```
@misc{santos2022triplebarrier,
  title = {Algorithmic trading: triple barrier labelling},
  year = {2022},
  url = {https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/},
  note = {Accessed: 2026-03-11}
}
```

If you prefer a version with @online for biblatex, use this instead.^[97]

```
@online{santos2022triplebarrier,
  title   = {Algorithmic trading: triple barrier labelling},
  year    = {2022},
  url     = {https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/},
  urldate = {2026-03-11}
}
```

✱

```
# <q>Split with a quarterly expanding window into train, validation, and out-of-sample t
```

It means you move through time in order, let the training set **expand** each quarter, and keep validation and test strictly in the future so there is no leakage.^{[99] [100]}

Example

Split	Train	Validation	OOS test
1	2022 Q1-Q4 ^[99] ^[100]	2023 Q1 ^[99]	2023 Q2 ^[99] ^[100]
2	2022 Q1-2023 Q1 ^[99] ^[100]	2023 Q2 ^[99]	2023 Q3 ^[99] ^[100]
3	2022 Q1-2023 Q2 ^[99] ^[100]	2023 Q3 ^[99]	2023 Q4 ^[99] ^[100]
4	2022 Q1-2023 Q3 ^[99] ^[100]	2023 Q4 ^[99]	2024 Q1 ^[99] ^[100]

How to read it

Each step adds one new quarter to training, then uses the next quarter for validation and the quarter after that as the true out-of-sample test.^[100] ^[99]

So the model is always trained on the past and judged on unseen future periods, which is the point of walk-forward time-series splitting.^[99]

A compact version is: train = all data up to quarter t, val = quarter t+1, test = quarter t+2.^[100] ^[99]

✱

im currently writing a small section about the bar construction phase of the pipeline. Can i formulate it as, when the model gets to sample a datapoint?

Not quite.^[109]

“When the model gets to sample a datapoint” sounds like the model chooses the observation, but in this pipeline the bars are constructed first during preprocessing, and only then do they become data points for feature engineering, labeling, and training.^[109]

Better phrasing

You could write one of these:

- “The bar construction phase defines how raw market data are aggregated into observations.”^[109]
- “In this phase, tick data are sampled into bars, and each bar becomes a candidate data point in the modeling pipeline.”^[109]
- “Bar construction determines when a new observation enters the dataset.”^[109]

Best distinction

So yes to the idea, but I would avoid saying “the model gets to sample.”^[109]

A cleaner formulation is: “the pipeline samples raw market activity into bars, which serve as the model’s input observations.”^[109]

✱

can i say that this process turns the raw data from yahoo finance into bars that act as observations for the model to train train on?

Yes — that is a good way to say it.^[110]

A slightly cleaner version is: “This process transforms the raw Yahoo Finance data into bars, which then serve as observations for the model to train on.”^[110]

Slight nuance

If you want to be a bit more precise, say “aggregates” or “samples” instead of “turns,” since the pipeline groups raw market data into bar-based observations.^[110]

So an even stronger version is: “The bar construction phase samples raw Yahoo Finance data into bars that serve as observations for downstream feature engineering, labeling, and model training.”^[110]

✱

is the bar construction the same as sampling methods?

Almost, but not exactly.^{[111] [112]}

Sampling method is the rule for when a new bar is formed, while **bar construction** is the overall process of turning raw market data into bars using that rule.^{[112] [111]}

So:

- Sampling method = time bars, volume bars, dollar bars, range bars, CUSUM-based sampling.^[112]
- Bar construction = apply one of those methods and aggregate the underlying data into OHLC-style observations.^{[111] [112]}

A good one-line formulation is: “Bar construction is the preprocessing step, and the sampling method is the specific rule used within that step.”^{[111] [112]}

✱

so i use sampling methods to construct the bars right?

Yes, exactly.^[122]

You use sampling methods to construct bars from the raw market data.^[122]

A clean sentence is: “Bars are constructed by applying different sampling methods to the raw data.”^[122]

Or, if you want slightly more detail: “The bar construction phase uses sampling methods such as time, volume, dollar, range, or CUSUM-based sampling to convert raw market data into bar-based observations.”^[122]

✱

can you find a good article about cusum filtering?

Yes — a good free article is Terbe’s “CUSUM Filter”.^[123]

It gives the core intuition, the equations, and a simple implementation, so it is very good if you want to understand how the threshold and reset logic work.^[123]

Best pick

- Terbe — “CUSUM Filter”: best for understanding the mechanics from scratch.^[123]
<https://terbe.dev/notes/cusum-filter>^[123]

Good finance-specific backup

- Hudson & Thames — “Machine Learning Trading Essentials (Part 2)”: better if you want CUSUM explained inside the financial ML pipeline together with event filtering and triple-barrier labeling.
<https://hudsonthames.org/machine-learning-trading-essentials-part-2-fractionally-differentiated-features-filtering-and-labelling/>

For your thesis section, I would cite Hudson & Thames if you want pipeline context, and Terbe if you want the clearest standalone explanation.^[123]

✱

can you find an academic article? i have dtu find it so i should be able to find the article

Yes — if you want an academic source on CUSUM for finance, the best fit is probably: Yi, Coleman, and Ren (2006), “CUSUM method in predicting regime shifts and its performance in different stock markets allowing for transaction fees,” *Journal of Applied Statistics*, 33(7), 647–661.^{[139] [140]}

It is directly about applying CUSUM to financial markets, compares performance across stock

markets, and explicitly considers transaction fees, so it fits a trading-pipeline discussion better than a generic statistics paper.^[140] ^[139]

Also useful

If you want the original academic source behind CUSUM itself, use Page (1954), “Continuous Inspection Schemes,” *Biometrika*, 41(1–2), 100–115.^[141] ^[142]

If you want a more modern finance paper, there is also Astill et al. (2023), “CUSUM-Based Monitoring for Explosive Episodes in Financial Data in the Presence of Time-Varying Volatility,” *Journal of Financial Econometrics*, 21(1), 187–227.^[143] ^[144]

Best choice

For your thesis, I would cite Yi et al. (2006) if the point is “CUSUM applied in financial markets,” and Page (1954) if the point is “origin of the CUSUM method.”^[142] ^[139]

In DTU Findit, search exactly: “CUSUM method in predicting regime shifts” or “Continuous Inspection Schemes Page 1954”.^[139] ^[142]

✱✱

1. crypto.pdf
2. <https://en.wikipedia.org/wiki/CUSUM>
3. <https://dataopsschool.com/blog/cusum/>
4. crypto.pdf
5. <https://sixsigmastudyguide.com/cumulative-sum-chart-cusum/>
6. <https://tjuvblog-production.up.railway.app/blog/the-cusum-algorithm-all-the-essential-information-you-need-with-python-examples.mdx>
7. <https://github.com/giobbu/CUSUM>
8. <https://www.mathworks.com/help/signal/ref/cusum.html>
9. <https://blog.stackademic.com/the-cusum-algorithm-all-the-essential-information-you-need-with-python-examples-f6a5651bf2e5?gi=f3748814e406>
10. <https://de.mathworks.com/help/signal/ref/cusum.html>
11. <https://dataprocessing.aixcape.org/Algorithms/CUSUM/index.html>
12. <https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm>
13. <https://paperswithbacktest.com/wiki/information-driven-bars-table-rows>
14. crypto.pdf
15. <https://github.com/jzajpt/id-bars>
16. <https://www.comp.nus.edu.sg/~lowkh/pubs/icra2013.pdf>
17. <https://onepagecode.substack.com/p/quantscope-algorithmic-high-frequency>
18. <https://www.amazon.science/publications/adaptive-sampling-for-probabilistic-forecasting-under-distribution-shift>

19. <https://static1.squarespace.com/static/5230e9f8e4b06ab69d1d8068/t/64d18c8ea4ecc25b4c57e976/1691454607289/High-Resolution+Random+Control+at+Low+Frequencies.pdf>
20. <https://www.quantbeckman.com/p/what-are-your-bars-hiding-from-you>
21. <http://arxiv.org/pdf/2302.11870.pdf>
22. <https://arc.aiaa.org/doi/10.2514/6.2025-1468>
23. <https://www.nature.com/articles/s41598-025-22926-y>
24. <https://www.sciencedirect.com/science/article/abs/pii/S0167691125002002>
25. <https://arxiv.org/pdf/2110.11156.pdf>
26. <https://dl.acm.org/doi/10.1016/j.eswa.2022.119207>
27. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12578950/>
28. <https://arxiv.org/html/2407.05691v1>
29. crypto.pdf
30. crypto.pdf
31. crypto.pdf
32. crypto.pdf
33. crypto.pdf
34. crypto.pdf
35. <https://milvus.io/ai-quick-reference/what-is-feature-scaling-and-why-is-it-necessary-when-working-with-datasets>
36. <https://www.appliedaicourse.com/blog/feature-scaling-in-machine-learning/>
37. <https://www.projectpro.io/article/feature-scaling-in-machine-learning/990>
38. crypto.pdf
39. https://dev.to/vikas_gulia/feature-scaling-in-machine-learning-why-it-matters-and-how-to-do-it-45m5
40. https://en.wikipedia.org/wiki/Feature_scaling
41. <https://www.youtube.com/watch?v=gb1frohFFgM>
42. https://scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html
43. <https://www.geeksforgeeks.org/machine-learning/feature-engineering-scaling-normalization-and-standardization/>
44. <https://datasciencedojo.com/blog/feature-scaling/>
45. <https://www.geeksforgeeks.org/machine-learning/ml-feature-scaling-part-1/>
46. crypto.pdf
47. crypto.pdf
48. crypto.pdf
49. crypto.pdf
50. crypto.pdf
51. <https://github.com/nkonts/barrier-method>
52. <https://www.wiley.com/en-us/Advances+in+Financial+Machine+Learning-p-9781119482086>

53. <https://hudsonthames.org/does-meta-labeling-add-to-signal-efficacy-triple-barrier-method/>
54. <https://github.com/braverock/quantstrat/issues/105>
55. https://www.reddit.com/r/algorithmtrading/comments/g08o22/function_to_compute_the_triple_barrier_method/
56. <https://www.youtube.com/watch?v=mC53d7QhxEw>
57. https://www.reddit.com/r/quantfinance/comments/1p47ux5/triple_barrier_labeling_in_production_lópez_de/
58. <https://ayratmurtazin.beehiiv.com/p/labeling-stock-prices-for-ml-with-triple-barrier-methods>
59. <http://oreateai.com/blog/beyond-fixed-rules-understanding-the-dynamic-triple-barrier-method/14f01a5eb25348614a09f93467819782>
60. https://www.reddit.com/r/algorithmtrading/comments/1c2jtut/creative_target_variables_for_supervised_ml/
61. https://mlfinpy.readthedocs.io/en/stable/_modules/mlfinpy/labeling/labeling.html
62. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3104847
63. [https://agorism.dev/book/finance/ml/Marcos Lopez de Prado - Advances in Financial Machine Learning-Wiley \(2018\).pdf](https://agorism.dev/book/finance/ml/Marcos Lopez de Prado - Advances in Financial Machine Learning-Wiley (2018).pdf)
64. <https://www.adlibris.com/sv/bok/advances-in-financial-machine-learning-9781119482086>
65. <https://www.bokus.com/bok/9781119482086/advances-in-financial-machine-learning/>
66. <https://books.google.com/books?id=oU9KDwAAQBAJ&printsec=frontcover>
67. <https://www.goodreads.com/en/book/show/36703132-advances-in-financial-machine-learning>
68. https://play.google.com/store/books/details/Advances_in_Financial_Machine_Learning?id=c75IDwAAQBAJ&hl=ja
69. https://openlibrary.org/books/OL26950595M/Advances_in_financial_machine_learning
70. <https://www.wob.com/en-gb/books/marcos-lopez-de-prado/advances-in-financial-machine-learning/9781119482086>
71. <https://www.econbiz.de/Record/advances-in-financial-machine-learning-lópez-prado-marcos/10011841464>
72. https://play.google.com/store/books/details/Advances_in_Financial_Machine_Learning?id=vORKDwAAQBAJ&hl=es_US
73. <https://nalda.ch/products/9781119482086>
74. <https://www.goodreads.com/book/show/36703132-advances-in-financial-machine-learning>
75. https://reasonabledeviations.com/notes/adv_fin_ml/
76. <https://www.newsletter.quantreo.com/p/the-triple-barrier-labeling-of-marco>
77. <https://blog.quantinsti.com/triple-barrier-method-gpu-python/>
78. <https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/>
79. <https://wire.insiderfinance.io/triple-barrier-method-4cb60cf4c2f0?gi=cf63848edb32>
80. <https://github.com/maxzager/Financial-series-and-Triple-Barrier-Method>
81. [crypto.pdf](#)
82. <https://github.com/maxzager/Financial-series-and-Triple-Barrier-Method>
83. <https://williamsantos.me/posts/2022/triple-barrier-labelling-algorithm/>
84. <https://courses.ems.psu.edu/earth107/node/1088>

85. <https://pypi.org/project/triple-barrier/>
86. <https://github.com/nkonts/barrier-method>
87. <https://www.thamesclippers.com/blog/thames-barrier>
88. <https://wire.insiderfinance.io/triple-barrier-method-4cb60cf4c2f0>
89. <https://www.gov.uk/guidance/the-thames-barrier>
90. https://en.wikipedia.org/wiki/Thames_Barrier
91. <https://www.ice.org.uk/what-is-civil-engineering/infrastructure-projects/thames-barrier>
92. <https://www.youtube.com/watch?v=yM2KOIz7CNI>
93. <https://www.environmentandsociety.org/arcadia/thames-barrier-londons-moveable-flood-defense>
94. <https://blog.quantinsti.com/triple-barrier-method-gpu-python/>
95. https://www.reddit.com/r/quantfinance/comments/1p47ux5/triple_barrier_labeling_in_production_lópez_de/
96. <https://www.youtube.com/watch?v=JRuCy-z7FYI>
97. crypto.pdf
98. image.jpg
99. <https://algotrading101.com/learn/train-test-split/>
100. <https://stackoverflow.com/questions/58069691/how-to-create-a-train-test-split-of-time-series-data-by-year>
101. <https://www.youtube.com/watch?v=ImjcVcwjvUg>
102. <https://speedbot.tech/blog/algo-trading-4/understanding-timeseries-in-algo-trading-103>
103. <https://www.youtube.com/watch?v=9yQTpp6L4VA>
104. <https://themindfulmeld.substack.com/p/how-to-effectively-split-data-for>
105. <https://www.youtube.com/watch?v=ulaA-fz6oz4>
106. <https://hatchjs.com/time-series-split-sklearn/>
107. https://www.reddit.com/r/algotrading/comments/vn0xsg/traintest_split/
108. <https://www.youtube.com/watch?v=xPWz1reXeSA>
109. crypto.pdf
110. crypto.pdf
111. <https://www.quantbeckman.com/p/what-are-your-bars-hiding-from-you>
112. crypto.pdf
113. https://www.broda.co.uk/doc/QMC_Basket_AAD.pdf
114. <https://www.analystforum.com/t/difference-b-w-optimization-and-stratified-sampling/40278>
115. <https://www.bakertilly.com/insights/construction-audit-resource-center>
116. <https://alpaca.markets/learn/alternative-bars-01/>
117. https://www.linkedin.com/posts/suyash-verma-88502b238_machinelearning-financialml-quantfinance-activity-7415098125356314626-7HX4
118. <https://pcaobus.org/oversight/standards/auditing-standards/details/AS2315>

119. https://fliphtml5.com/fzqli/zwcp/Advances_in_Financial_Machine_Learning-Marcos_Lopez_de_Prado/53/
120. <https://haas.berkeley.edu/wp-content/uploads/standarderror.pdf>
121. <https://analystprep.com/cfa-level-1-exam/quantitative-methods/resampling/>
122. [crypto.pdf](#)
123. [crypto.pdf](#)
124. <http://momentum.technicalanalysis.org.uk/LaYa97.pdf>
125. <https://arxiv.org/abs/2210.17312>
126. <https://www.youtube.com/watch?v=iADIO-bEA2Q>
127. <http://www.technicalanalysis.org.uk/momentum/LaYa97.pdf>
128. <https://jmlr.org/papers/v24/21-t230.html>
129. <https://mlfinpy.readthedocs.io/en/latest/Filtering.html>
130. <https://www.youtube.com/watch?v=f6mfoyejC-8>
131. https://www.reddit.com/r/quant/comments/1khn7el/cusum_filter_is_it_effective_and_why/
132. <https://hudsonthames.org/machine-learning-trading-essentials-part-2-fractionally-differentiated-features-filtering-and-labelling/>
133. <https://algofin.substack.com/p/algorithmic-trading-models-cumulative>
134. <https://github.com/muMAJJI/Trading---CUSUM-FILTER>
135. <https://hudsonthames.org/category/advances-in-financial-machine-learning/>
136. <https://www.marketcalls.in/amibroker/cusum-filter-strategy-in-amibroker-afl-a-practical-implementation.html>
137. <https://www.jmlr.org/papers/volume24/21-t230/21-t230.pdf>
138. <https://terbe.dev/notes/cusum-filter>
139. <https://eprints.ncl.ac.uk/72242>
140. <https://www.econbiz.de/Record/cusum-method-in-predicting-regime-shifts-and-its-performance-in-different-stock-markets-allowing-for-transaction-fees/10005458361>
141. <https://www.sciepub.com/reference/429380>
142. <https://academic.oup.com/biomet/issue/41/1-2>
143. <https://academic.oup.com/jfec/article/21/1/187/6268988>
144. <https://academic.oup.com/jfec/article/21/1/187/6268988?searchresult=1>
145. https://econpapers.repec.org/article/oupjfinec/v_3a21_3ay_3a2023_3ai_3a1_3ap_3a187-227.htm
146. <https://hudsonthames.org/category/advances-in-financial-machine-learning/>
147. https://meetings.ams.org/math/fall2020c/mediafile/Handout/Paper1058/4107903_AMSEIPASO_try.pdf
148. <https://dialnet.unirioja.es/descarga/articulo/3682765.pdf>
149. <http://www.technicalanalysis.org.uk/momentum/LaYa97.pdf>
150. <http://momentum.technicalanalysis.org.uk/LaYa97.pdf>
151. <https://www.stat.cmu.edu/technometrics/90-00/vol-42-01/v4201102.pdf>

152. <https://hudsonthames.org/machine-learning-trading-essentials-part-2-fractionally-differentiated-features-filtering-and-labelling/>
153. <https://pub.towardsai.net/algorithmic-trading-models-cumulative-sums-filter-e2200ff45ef1>
154. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8794397/>
155. <http://investicsfiles.s3.amazonaws.com/Documentation/Using+Statistical+Process+Control+To+Monitor+Active+Managers.pdf>
156. <https://ideas.repec.org/a/taf/japsta/v33y2006i7p647-661.html>
157. <https://www.scribd.com/document/978357075/Cusum-Techniques-for-Technical-Trading-in-Financia>
158. <https://www.statistics.gov.hk/wsc/CPS009-P7-S.pdf>
159. <https://www.jstor.org/stable/2333307>
160. <https://isi-web.org/sites/default/files/import/files-2013/CPS009-P7-S.pdf>
161. <https://scholar.lib.vt.edu/ejournals/SNDE/002/articles/v2n2001.pdf>
162. <https://methods.stat.kit.edu/download/IFORS2014.pdf>
163. <https://zh.mindat.org/reference.php?id=9342733>
164. <https://ideas.repec.org/a/oup/jfinec/v21y2023i1p187-227.html>
165. <https://repository.essex.ac.uk/30046/>
166. <https://academic.oup.com/jfec/advance-article-abstract/doi/10.1093/jfinec/nbab009/6268988>
167. <https://scholar.google.se/citations?user=8W5h0aUAAAAJ&hl=th>
168. <https://scholar.google.com/citations?user=CylcD9kAAAAJ&hl=en>
169. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7712961/>
170. <https://nottingham-repository.worktribe.com/orgunit/62411/1100-economics/outputs?Author=DAVID+HARVEY&Year=2021&page=1>
171. <https://repository.essex.ac.uk/30046/10/nbab009.pdf>
172. <https://www.semanticscholar.org/paper/CUSUM-Based-Monitoring-for-Explosive-Episodes-in-in-Astill-Harvey/4fbefdbe3f0bdcdd50d684c6f6badd726f724987>
173. <https://www.econbiz.de/Record/cusum-based-monitoring-for-explosive-episodes-in-financial-data-in-the-presence-of-time-varying-volatility-astill-sam/10013542862>
174. <https://www.sciencedirect.com/science/article/abs/pii/S2452306223000540>
175. <https://www.ostirion.net/post/cusum-event-identification-not-just-for-sampling>
176. <https://nottingham-repository.worktribe.com/index.php/output/5396026/cusum-based-monitoring-for-explosive-episodes-in-financial-data-in-the-presence-of-time-varying-volatility>
177. https://www.math.cmu.edu/users/ccf/docs/seminar_6s/Hadjiladis.pdf